

LEC 7

27th July, 2026

Data Structures and Algorithms — Stacks (Using Structs in C++)

Topic Coverage: Stack ADT · Applications · Implementation Strategies · C++ `struct` Fundamentals · Array-based Stack using Struct · Linked-List-based Stack using Struct · Complexity Analysis · Common Interview Problems

1. Introduction to Stacks

A **Stack** is a linear data structure that follows a particular order in which operations are performed. The order is **LIFO — Last In, First Out**. This means the element inserted last is the one removed first.

Key Characteristics

- Stack is an **ordered collection** of items.
- Insertion and deletion happen only from **one end**, called the **top**.
- It is a **restricted-access** data structure — unlike arrays, you cannot access an arbitrary element directly; you can only access the top.
- Stacks can be implemented using **arrays**, **linked lists**, or **dynamic arrays** (like `std::vector`).

Formal Definition

A stack is an Abstract Data Type (ADT) that supports two principal operations:

- `push(x)` — insert element `x` at the top.
- `pop()` — remove and return the element at the top.

Along with a few auxiliary operations: `peek()` / `top()`, `isEmpty()`, `isFull()` (for bounded implementations), and `size()`.

2. Real-World Analogies

Analogy	Explanation
Stack of plates	You can only add/remove a plate from the top of the pile.
Browser back button	The last page visited is the first one you go back to.
Undo feature in editors	The most recent action is undone first.
Stack of books	You pick up the topmost book first.
Function call stack	The most recently called function returns first.

3. Stack Operations

Operation	Description	Time Complexity
<code>push(x)</code>	Insert element <code>x</code> on top of the stack	$O(1)$
<code>pop()</code>	Remove the top element and return it	$O(1)$
<code>peek()</code> / <code>top()</code>	Return the top element without removing it	$O(1)$
<code>isEmpty()</code>	Check whether the stack has zero elements	$O(1)$
<code>isFull()</code>	Check whether the stack has reached capacity (array-based only)	$O(1)$
<code>size()</code>	Return the current number of elements	$O(1)$

Exceptions to Handle

- **Stack Overflow** — attempting `push()` when the stack is at maximum capacity (relevant only for fixed-size array implementations).
 - **Stack Underflow** — attempting `pop()` or `peek()` on an empty stack.
-

4. Applications of Stacks

1. Expression Evaluation & Conversion

- Infix to Postfix / Prefix conversion
- Evaluation of Postfix and Prefix expressions

2. Balanced Parentheses / Bracket Matching — validating `{[()]}` style expressions.

3. **Function Call Management** — the **call stack** in program execution (recursion is implemented via an implicit stack).
 4. **Undo/Redo functionality** in text editors and software.
 5. **Backtracking algorithms** — maze solving, N-Queens, DFS traversal of graphs/trees.
 6. **Browser history navigation**.
 7. **Syntax Parsing** in compilers (e.g., checking nested structures).
 8. **Reversing data** — reversing a string, a linked list, or the order of elements in a queue.
 9. **Memory management** — stack memory for local variables and function frames.
-

5. Structs in C++ — Fundamentals

A **struct** (structure) in C++ is a user-defined data type that groups together variables of **different data types** under a single name. It is one of the fundamental building blocks used to represent a **node** or a **record** in data structures.

5.1 Syntax

```
struct StructName {  
    dataType member1;  
    dataType member2;  
    // ... more members  
};
```

5.2 Basic Example

```
#include <iostream>  
using namespace std;  
  
struct Student {  
    int rollNumber;  
    string name;  
    float marks;  
};  
  
int main() {  
    Student s1;           // declaring a struct variable  
    s1.rollNumber = 101;  
    s1.name = "Aditi";  
    s1.marks = 89.5;  
  
    cout << s1.rollNumber << " " << s1.name << " " << s1.marks << endl;
```

```
    return 0;
}
```

5.3 Ways to Initialize a Struct

```
struct Point {
    int x;
    int y;
};

// Method 1: Member-wise assignment
Point p1;
p1.x = 5;
p1.y = 10;

// Method 2: Aggregate initialization
Point p2 = {5, 10};

// Method 3: Brace initialization (C++11 onward)
Point p3{5, 10};

// Method 4: Using 'new' for dynamic allocation
Point* p4 = new Point{5, 10};
```

5.4 Struct with Member Functions (Modern C++)

In modern C++, structs can contain **member functions**, **constructors**, and even **access specifiers** — a struct is essentially a class with default `public` access.

```
struct Point {
    int x, y;

    // constructor
    Point(int a, int b) : x(a), y(b) {}

    // member function
    void display() {
        cout << "(" << x << ", " << y << ")" << endl;
    }
};

int main() {
    Point p(3, 4);
    p.display(); // (3, 4)
}
```

```
    return 0;
}
```

5.5 Self-Referential Structs (Critical for Data Structures)

A struct can contain a **pointer to its own type**. This concept is the foundation of **linked lists**, **stacks**, **queues**, and **trees**.

```
struct Node {
    int data;
    Node* next;    // self-referential pointer
};
```

This single idea — a struct holding a pointer to another struct of the same type — is what allows us to build **dynamically linked chains of nodes** in memory, which is exactly how a linked-list-based stack works.

5.6 Struct Arrays

```
struct Point {
    int x, y;
};

Point arr[5];    // array of 5 Point structs
arr[0].x = 1;
arr[0].y = 2;
```

This is the foundation for the **array-based stack implementation** shown later — instead of storing raw integers, we can store an array of structs.

6. Struct vs Class in C++

Feature	struct	class
Default access specifier	public	private
Default inheritance	public	private
Typical usage convention	Plain data holders (POD types), nodes	Full objects with encapsulation/behavior

Feature	struct	class
Functionality	Can have constructors, destructors, member functions, operator overloading (C++ only)	Same
Memory layout	Identical to class (no runtime difference)	Identical to struct

Exam Tip: In C++ (unlike C), `struct` and `class` are functionally almost identical — the *only* real difference is the **default access specifier** and **default inheritance mode**.

7. Stack Implementation Using Array + Struct

This is the most common way UG-level DSA courses introduce stacks — a **fixed-size array** wrapped inside a **struct**, with an integer `top` tracking the index of the topmost element.

7.1 Design

```
struct Stack {
    int arr[MAX];    // fixed-size storage
    int top;         // index of top element (-1 means empty)
};
```

7.2 Full Working Code

```
#include <iostream>
using namespace std;

#define MAX 100    // maximum capacity of the stack

// Struct representing the Stack data structure
struct Stack {
    int arr[MAX];
    int top;
};

// Initialize the stack
void initStack(Stack &s) {
    s.top = -1;    // -1 indicates an empty stack
}

// Check if stack is empty
bool isEmpty(Stack &s) {
```

```

    return s.top == -1;
}

// Check if stack is full
bool isFull(Stack &s) {
    return s.top == MAX - 1;
}

// Push operation
void push(Stack &s, int value) {
    if (isFull(s)) {
        cout << "Stack Overflow! Cannot push " << value << endl;
        return;
    }
    s.arr[++s.top] = value; // increment top, then insert
    cout << value << " pushed onto stack." << endl;
}

// Pop operation
int pop(Stack &s) {
    if (isEmpty(s)) {
        cout << "Stack Underflow! Cannot pop." << endl;
        return -1; // sentinel error value
    }
    return s.arr[s.top--]; // return top, then decrement
}

// Peek / Top operation
int peek(Stack &s) {
    if (isEmpty(s)) {
        cout << "Stack is empty." << endl;
        return -1;
    }
    return s.arr[s.top];
}

// Print the entire stack (top to bottom)
void display(Stack &s) {
    if (isEmpty(s)) {
        cout << "Stack is empty." << endl;
        return;
    }
    cout << "Stack (top -> bottom): ";
    for (int i = s.top; i >= 0; i--) {
        cout << s.arr[i] << " ";
    }
}

```

```

        cout << endl;
    }

    int main() {
        Stack s;
        initStack(s);

        push(s, 10);
        push(s, 20);
        push(s, 30);
        display(s);

        cout << "Top element: " << peek(s) << endl;

        cout << "Popped: " << pop(s) << endl;
        display(s);

        return 0;
    }

```

7.3 Expected Output

```

10 pushed onto stack.
20 pushed onto stack.
30 pushed onto stack.
Stack (top -> bottom): 30 20 10
Top element: 30
Popped: 30
Stack (top -> bottom): 20 10

```

7.4 Step-by-Step Trace Table

Operation	top before	arr state	top after
initStack	—	[]	-1
push(10)	-1	[10]	0
push(20)	0	[10, 20]	1
push(30)	1	[10, 20, 30]	2
pop() returns 30	2	[10, 20]	1

7.5 Why Pass Stack &s by Reference?

Since `Stack` is a struct, passing it **by value** would copy the entire array (100 integers) on every function call, and any modification (like updating `top`) would be lost after the function returns. Passing **by reference** (`&`) avoids copying and lets functions directly modify the caller's struct instance.

8. Stack Implementation Using Linked List + Struct

Array-based stacks have a fixed capacity. A **linked-list-based stack** grows and shrinks dynamically, limited only by available heap memory. Here we use **two structs**: one for a `Node`, and one for the `Stack` itself (holding a pointer to the top node).

8.1 Design

```
struct Node {
    int data;
    Node* next;
};

struct Stack {
    Node* top;    // pointer to the topmost node
    int count;    // optional: tracks number of elements
};
```

8.2 Full Working Code

```
#include <iostream>
using namespace std;

// Node struct - represents a single element in the stack
struct Node {
    int data;
    Node* next;
};

// Stack struct - holds a pointer to the top node
struct Stack {
    Node* top;
    int count;
};

// Initialize the stack
void initStack(Stack &s) {
```

```

    s.top = nullptr;
    s.count = 0;
}

// Check if stack is empty
bool isEmpty(Stack &s) {
    return s.top == nullptr;
}

// Push operation - O(1)
void push(Stack &s, int value) {
    Node* newNode = new Node(); // dynamically allocate a new node
    newNode->data = value;
    newNode->next = s.top; // new node points to the current top
    s.top = newNode; // update top to the new node
    s.count++;
    cout << value << " pushed onto stack." << endl;
}

// Pop operation - O(1)
int pop(Stack &s) {
    if (isEmpty(s)) {
        cout << "Stack Underflow! Cannot pop." << endl;
        return -1;
    }
    Node* temp = s.top;
    int poppedValue = temp->data;
    s.top = s.top->next; // move top pointer to the next node
    delete temp; // free memory of the removed node
    s.count--;
    return poppedValue;
}

// Peek / Top operation
int peek(Stack &s) {
    if (isEmpty(s)) {
        cout << "Stack is empty." << endl;
        return -1;
    }
    return s.top->data;
}

// Display the stack
void display(Stack &s) {
    if (isEmpty(s)) {
        cout << "Stack is empty." << endl;
    }
}

```

```

        return;
    }
    Node* temp = s.top;
    cout << "Stack (top -> bottom): ";
    while (temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

// Free all remaining nodes (important to avoid memory leaks)
void destroyStack(Stack &s) {
    while (!isEmpty(s)) {
        pop(s);
    }
}

int main() {
    Stack s;
    initStack(s);

    push(s, 5);
    push(s, 15);
    push(s, 25);
    display(s);

    cout << "Top element: " << peek(s) << endl;
    cout << "Popped: " << pop(s) << endl;
    display(s);

    destroyStack(s); // clean up remaining memory
    return 0;
}

```

8.3 Expected Output

```

5 pushed onto stack.
15 pushed onto stack.
25 pushed onto stack.
Stack (top -> bottom): 25 15 5
Top element: 25
Popped: 25
Stack (top -> bottom): 15 5

```

8.4 Visual Representation

```
push(5):    top -> [5|NULL]

push(15):   top -> [15|*] -> [5|NULL]

push(25):   top -> [25|*] -> [15|*] -> [5|NULL]

pop():      returns 25, top -> [15|*] -> [5|NULL]
```

8.5 Array-based vs Linked-List-based Stack

Feature	Array-based Stack	Linked-List-based Stack
Memory allocation	Static (fixed at compile time, or fixed max)	Dynamic (grows/shrinks at runtime)
Overflow possibility	Yes, if capacity exceeded	No (until heap memory is exhausted)
Memory overhead	None extra — just the array	Extra memory per node for the next pointer
Cache performance	Better (contiguous memory)	Worse (scattered memory, pointer chasing)
Implementation complexity	Simpler	Slightly more complex (manual memory management)
Access speed	$O(1)$, very fast due to indexing	$O(1)$, but with pointer dereferencing overhead

9. Dynamic (Resizable) Array Stack using Struct

A middle ground: use a struct that wraps a **dynamically allocated array** (via `new`), which is resized (usually doubled) when full. This mimics how `std::vector` works internally and avoids the fixed-capacity limitation of Section 7.

```
#include <iostream>
using namespace std;

struct DynamicStack {
    int* arr;
    int capacity;
    int top;
```

```

};

void initStack(DynamicStack &s, int initialCapacity = 4) {
    s.capacity = initialCapacity;
    s.arr = new int[s.capacity];
    s.top = -1;
}

bool isEmpty(DynamicStack &s) {
    return s.top == -1;
}

// Doubles the capacity when the stack is full
void resize(DynamicStack &s) {
    int newCapacity = s.capacity * 2;
    int* newArr = new int[newCapacity];

    for (int i = 0; i <= s.top; i++) {
        newArr[i] = s.arr[i];
    }

    delete[] s.arr;          // free old array
    s.arr = newArr;
    s.capacity = newCapacity;
    cout << "[Resized capacity to " << newCapacity << "]" << endl;
}

void push(DynamicStack &s, int value) {
    if (s.top == s.capacity - 1) {
        resize(s);
    }
    s.arr[++s.top] = value;
}

int pop(DynamicStack &s) {
    if (isEmpty(s)) {
        cout << "Stack Underflow!" << endl;
        return -1;
    }
    return s.arr[s.top--];
}

int peek(DynamicStack &s) {
    if (isEmpty(s)) return -1;
    return s.arr[s.top];
}

```

```

void destroyStack(DynamicStack &s) {
    delete[] s.arr;
    s.arr = nullptr;
}

int main() {
    DynamicStack s;
    initStack(s, 2);    // start small to demonstrate resizing

    push(s, 1);
    push(s, 2);
    push(s, 3);    // triggers resize
    push(s, 4);
    push(s, 5);    // triggers resize again

    while (!isEmpty(s)) {
        cout << pop(s) << " ";
    }
    cout << endl;

    destroyStack(s);
    return 0;
}

```

Amortized push complexity: $O(1)$ — even though a resize takes $O(n)$, it happens rarely enough that the *average* cost per push remains constant over a long sequence of operations.

10. Complexity Analysis Summary

Implementation	Push	Pop	Peek	Space Complexity
Fixed Array + Struct	$O(1)$	$O(1)$	$O(1)$	$O(n)$ — pre-allocated
Linked List + Struct	$O(1)$	$O(1)$	$O(1)$	$O(n)$ — grows as needed, plus pointer overhead
Dynamic Array + Struct	$O(1)$ amortized	$O(1)$	$O(1)$	$O(n)$ — grows as needed

11. Classic Stack Problems (with Code)

11.1 Balanced Parentheses

Problem: Given a string containing (,) , { , } , [,] , determine if the brackets are balanced.

```
#include <iostream>
#include <string>
using namespace std;

#define MAX 100

struct Stack {
    char arr[MAX];
    int top;
};

void initStack(Stack &s) { s.top = -1; }
bool isEmpty(Stack &s) { return s.top == -1; }
void push(Stack &s, char c) { s.arr[++s.top] = c; }
char pop(Stack &s) { return s.arr[s.top--]; }
char peek(Stack &s) { return s.arr[s.top]; }

bool isBalanced(string expr) {
    Stack s;
    initStack(s);

    for (char c : expr) {
        if (c == '(' || c == '{' || c == '[') {
            push(s, c);
        } else if (c == ')' || c == '}' || c == ']') {
            if (isEmpty(s)) return false;

            char top = pop(s);
            if (c == ')' && top != '(') return false;
            if (c == '}' && top != '{') return false;
            if (c == ']' && top != '[') return false;
        }
    }
    return isEmpty(s); // balanced only if stack is empty at the end
}

int main() {
    string expr = "{[(a+b)*(c-d)]}";
    cout << (isBalanced(expr) ? "Balanced" : "Not Balanced") << endl;
```

```
    return 0;
}
```

11.2 Infix to Postfix Conversion

Problem: Convert an infix expression (e.g., `a+b*c`) into postfix (e.g., `abc**`).

```
#include <iostream>
#include <string>
using namespace std;

#define MAX 100

struct Stack {
    char arr[MAX];
    int top;
};

void initStack(Stack &s) { s.top = -1; }
bool isEmpty(Stack &s) { return s.top == -1; }
void push(Stack &s, char c) { s.arr[++s.top] = c; }
char pop(Stack &s) { return s.arr[s.top--]; }
char peek(Stack &s) { return s.arr[s.top]; }

int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    if (op == '^') return 3;
    return 0;
}

bool isOperator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '^');
}

string infixToPostfix(string infix) {
    Stack s;
    initStack(s);
    string postfix = "";

    for (char c : infix) {
        if (isalnum(c)) {
            postfix += c; // operands go directly to output
        }
        else if (c == '(') {
```



```

        push(s, c);
    }
    else if (c == ')') {
        while (!isEmpty(s) && peek(s) != '(') {
            postfix += pop(s);
        }
        pop(s);    // discard the '('
    }
    else if (isOperator(c)) {
        while (!isEmpty(s) && precedence(peek(s)) >= precedence(c)) {
            postfix += pop(s);
        }
        push(s, c);
    }
}

while (!isEmpty(s)) {
    postfix += pop(s);
}

return postfix;
}

int main() {
    string infix = "a+b*c-d/e";
    cout << "Postfix: " << infixToPostfix(infix) << endl;
    return 0;
}

```

11.3 Evaluating a Postfix Expression

```

#include <iostream>
#include <string>
#include <cmath>
using namespace std;

#define MAX 100

struct Stack {
    int arr[MAX];
    int top;
};

void initStack(Stack &s) { s.top = -1; }
bool isEmpty(Stack &s) { return s.top == -1; }

```

```

void push(Stack &s, int v) { s.arr[++s.top] = v; }
int pop(Stack &s) { return s.arr[s.top--]; }

int evaluatePostfix(string expr) {
    Stack s;
    initStack(s);

    for (char c : expr) {
        if (isdigit(c)) {
            push(s, c - '0'); // convert char digit to int
        } else {
            int val2 = pop(s);
            int val1 = pop(s);
            switch (c) {
                case '+': push(s, val1 + val2); break;
                case '-': push(s, val1 - val2); break;
                case '*': push(s, val1 * val2); break;
                case '/': push(s, val1 / val2); break;
            }
        }
    }
    return pop(s);
}

int main() {
    string postfix = "23*4+"; // (2*3)+4 = 10
    cout << "Result: " << evaluatePostfix(postfix) << endl;
    return 0;
}

```

11.4 Reversing a String Using a Stack

```

#include <iostream>
#include <string>
using namespace std;

#define MAX 100

struct Stack {
    char arr[MAX];
    int top;
};

void initStack(Stack &s) { s.top = -1; }
void push(Stack &s, char c) { s.arr[++s.top] = c; }

```

```

char pop(Stack &s) { return s.arr[s.top--]; }
bool isEmpty(Stack &s) { return s.top == -1; }

string reverseString(string str) {
    Stack s;
    initStack(s);

    for (char c : str) push(s, c);

    string reversed = "";
    while (!isEmpty(s)) reversed += pop(s);

    return reversed;
}

int main() {
    cout << reverseString("MPSTME") << endl;    // EMTSPM
    return 0;
}

```

12. Common Mistakes & Exam Tips

1. **Off-by-one errors on `top`** — Remember: `top = -1` means empty for array-based stacks; the first `push` should place the element at index `0`.
 2. **Forgetting to check `isEmpty()` before `pop()` / `peek()`** — leads to undefined behavior (array-based) or a null-pointer dereference (linked-list-based).
 3. **Not freeing memory** — in linked-list-based stacks, always `delete` the popped node; otherwise it causes a memory leak.
 4. **Passing struct by value instead of by reference** — silently breaks all mutating operations because the function modifies a *copy*.
 5. **Confusing Stack Overflow (data structure) with Stack Overflow (runtime call-stack exhaustion from recursion)** — these are two different concepts that share a name.
 6. **In infix-to-postfix conversion**, forgetting to pop all remaining operators from the stack after processing the entire expression.
 7. **Struct vs Class confusion** — in C++, remember the only difference is the default access specifier (`public` for struct, `private` for class).
-

13. Practice Questions

1. Implement a stack using a **struct with a fixed-size array** that stores `float` values instead of `int`, and add an `average()` function that returns the average of all elements without modifying the stack (hint: use a temporary array/copy).
 2. Write a function `bool isPalindrome(string str)` using **two stacks** or one stack, implemented via struct-based linked list.
 3. Implement **two stacks in a single array** (space-efficient technique) using a struct: one stack grows from index `0` upward, the other grows from index `n-1` downward.
 4. Design a struct-based stack that also supports a `getMin()` operation in $O(1)$ time (hint: use an **auxiliary stack** that tracks the minimum at each level).
 5. Convert a given **infix expression to prefix** notation using a stack (reverse the string, swap brackets, convert to postfix, then reverse the result).
 6. Simulate **recursion using an explicit stack** (struct-based) instead of relying on the language's call stack — e.g., iterative factorial or iterative tree traversal.
 7. Given a stack, write a function to **sort the stack** using only one additional stack (no arrays or other data structures).
 8. Implement the **Next Greater Element** problem for an array using a stack.
-

Summary

- A **Stack** is a **LIFO** data structure supporting `push`, `pop`, `peek`, `isEmpty` in $O(1)$ time.
- A **struct** in C++ groups related variables (and, in modern C++, functions) under one type — the natural container for a stack's internal state (`array + top`, or `Node* top`).
- **Self-referential structs** (`struct Node { data; Node* next; }`) are the building block for linked-list-based stacks.
- Stacks can be implemented via a **fixed-size array**, a **dynamically resizing array**, or a **linked list**, each with distinct trade-offs in memory usage and flexibility.
- Classic applications include **balanced parentheses checking**, **expression conversion/evaluation**, and **backtracking/recursion simulation**.